

OSDHack 2026

Resource Guide

On-Device AI / TinyML / Embedded Models and similar project families

Build AI that runs closer to the user: faster, lighter, more private, and open source.

Theme	On Device AI
Focus	Local models, browser AI, mobile AI, edge AI, TinyML, embedded ML
Core rule	The main AI functionality must run locally, on-device, in-browser, on edge hardware, or on embedded hardware.
Cloud support	Fine for hosting, auth, dashboards, email alerts, data sync, preparation, training, and export.
Official support	Discord and Unstop
Organizer	Open Source Developers Community (OSDC)



Contents

1	Core Philosophy	1
2	Models vs. Libraries	1
3	Open Model Examples: The Weights	1
3.1	Local LLMs: Text and Code.	1
3.2	Vision-Language and Multimodal.	1
3.3	Audio, Speech, and Voice	2
3.4	Computer Vision and Object Detection	2
3.5	Embeddings: Local RAG and Semantic Search	2
4	Execution Frameworks and Compute Environments.	2
4.1	Where to Run and Deploy Locally	2
4.2	Where to Build, Train, and Test for Free	2
5	Training and Optimization Tools	2
5.1	Fine-Tuning	3
5.2	Quantization.	3
5.3	Data Sources	3
6	Simple Project Notes to Include.	3
7	Quick Links Directory	3

1 Core Philosophy

Use the smallest model that solves the problem well. OSDHack 2026 is about local, practical AI: a carefully fine-tuned, quantized, and deployed 1.5B model can be a better fit than a much larger model that only runs on heavy hardware. The goal is not to avoid the internet entirely; the goal is to keep the main AI work on the user's device, browser, edge board, phone, laptop, or embedded hardware.

Important Nuance: The On-Device Rule

Your entire project does not have to be 100% offline. Internet access is fine for secondary features such as sending an email alert, pulling weather data, logging in, hosting a dashboard, syncing stats, or preparing/exporting a model. The main AI functionality should run locally using an on-device model.

2 Models vs. Libraries

A coding library or neural-network architecture template is not the same thing as a standalone local model. A library gives you tools; a model gives you trained weights or a runnable artifact.

Quick Clarification

Not a local model by itself	Importing a generic architecture such as <code>torchvision.models.resnet18</code> , or writing a blank CNN/LSTM/Transformer layer stack in PyTorch, Keras, or another framework.
Counts as a local model	A specific downloadable, trainable, or locally executable checkpoint or weights artifact, such as a <code>.gguf</code> , <code>.tflite</code> , <code>.onnx</code> , local model directory, or a specific weights repository on Hugging Face.

When you use a model, note its name, source link, license, file format, approximate size, and local runtime. If you converted, fine-tuned, or quantized it, mention the before/after formats briefly.

3 Open Model Examples: The Weights

These are starting points, not required choices. Pick models that match your use-case, license needs, device limits, and local runtime.

3.1 Local LLMs: Text and Code

- [Qwen](#): small 0.5B to 3B variants for local assistants, chat, and code tools.
- [Gemma](#): compact open models for efficient local deployment.
- [Microsoft Phi models](#): Phi-3, Phi-4-mini, and related small models.
- [SmolLM2](#): lightweight language models for smaller experiments.

Larger models such as Llama 3 8B can still be useful, but they usually need stronger laptops or specialized AI boards.

3.2 Vision-Language and Multimodal

- [Moondream2](#): lightweight image understanding for captions and visual Q&A.
- [Florence-2](#): captioning, visual tasks, and bounding-box style workflows.

3.3 Audio, Speech, and Voice

- [Whisper](#): tiny/base variants for offline speech-to-text.
- [Silero VAD](#): efficient voice activity detection for wake-word, meeting, or audio-triggered tools.
- Tiny keyword spotting models can be trained and exported through [Edge Impulse](#).

3.4 Computer Vision and Object Detection

- [Ultralytics YOLO](#): nano/small variants such as YOLOv8n or YOLO11n for local object detection.
- [MobileNet](#): efficient image classification backbone.
- Specific pre-trained ResNet or vision checkpoints exported to ONNX, LiteRT/TFLite, or another local format.

3.5 Embeddings: Local RAG and Semantic Search

- [all-MiniLM-L6-v2](#): compact sentence embeddings for private search.
- [BGE-small](#): small embedding model for retrieval workflows.

4 Execution Frameworks and Compute Environments

Instead of hosting the model on a cloud server, choose a runtime based on the local hardware where the project will run.

4.1 Where to Run and Deploy Locally

Target	Useful runtimes and notes
Desktops and single-board computers	llama.cpp for GGUF LLMs, ONNX Runtime for exported models, and OpenVINO for Intel hardware. Ollama and LM Studio are useful for local prototyping.
Web browsers	Transformers.js , ONNX Runtime Web , and WebLLM can run models in the user's browser through WebAssembly, WebGL, or WebGPU.
Mobile devices	LiteRT , ExecuTorch , and MediaPipe support Android/iOS, camera, voice, and perception tasks.
Microcontrollers and TinyML	Edge Impulse and TensorFlow Lite for Microcontrollers are practical for ESP32, Arduino Nano 33 BLE, sensors, keyword spotting, and tiny classifiers.

4.2 Where to Build, Train, and Test for Free

If your local device struggles to compile, fine-tune, or convert models, use cloud notebooks only for preparation, then export the model back to the local target.

- [Google Colab](#): useful for quick prototyping, small fine-tuning runs, and quantization tests.
- [Kaggle Notebooks](#): useful for dataset-heavy experiments and accelerator-backed notebooks when available.

5 Training and Optimization Tools

Adapting a model to your use-case can be as simple as fine-tuning a small classifier, exporting a detector, or converting an LLM to a smaller quantized format.

5.1 Fine-Tuning

- [Unsloth](#): faster fine-tuning workflows for supported small LLMs.
- [PEFT](#): LoRA/QLoRA adapters that train small adapter files instead of full model weights.
- Vision projects can fine-tune small detectors/classifiers on a focused dataset and export to ONNX, LiteRT/TFLite, or another local format.

5.2 Quantization

Quantization reduces model size and memory usage by lowering precision, often from FP16/FP32 to INT8 or INT4.

- [llama.cpp quantization](#): common for GGUF LLM workflows.
- [AutoAWQ](#): quantization workflows for supported LLMs.
- [ONNX Runtime quantization](#): useful for ONNX models.
- [TensorFlow Model Optimization Toolkit](#): useful for TensorFlow/LiteRT pipelines.

5.3 Data Sources

Useful starting points include [Roboflow Universe](#) for vision datasets, [Hugging Face Datasets](#) for text/audio/vision datasets, [Kaggle Datasets](#), and your own privacy-safe real-world sensor, audio, image, or document data.

6 Simple Project Notes to Include

Keep documentation short but reproducible. Mention the target hardware, model name, source link, license, runtime, setup command, and whether the model was fine-tuned, converted, or quantized.

Useful Examples

- **Target hardware:** Runs locally on an ESP32-S3, Android phone, Raspberry Pi, laptop CPU, or browser via WebGPU.
- **Model footprint:** Base model was about 3 GB; INT4 version is about 750 MB.
- **Performance:** 40 ms per frame, 12 FPS, 18 tokens/sec, or another simple local measurement.
- **Customization:** Fine-tuned YOLOv8n on 400 task-specific images, or exported a custom classifier to TFLite/ONNX.

7 Quick Links Directory

- [OSDHack Discord](#)
- [OSDHack Unstop listing](#)
- [OSI license list](#)
- [Hugging Face Models](#)
- [Transformers.js](#)
- [ONNX Runtime](#)
- [ONNX Runtime Web](#)
- [WebLLM](#)
- [llama.cpp](#)
- [Ollama docs](#)
- [LiteRT](#)
- [MediaPipe](#)

- [ExecuTorch](#)
- [TensorFlow Lite for Microcontrollers](#)
- [Edge Impulse](#)
- [Google Colab](#)
- [Kaggle Notebooks](#)
- [Roboflow Universe](#)
- [Unsloth](#)
- [PEFT](#)